

[illegible]

1. To find second smallest element in an array

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, i, smallest, secsmallest;
```

```
    printf("Enter the number: ");
```

```
    scanf("%d", &n);
```

```
    if (n < 2) {
```

```
        printf("Array must have 2 elements");
```

```
        return 1;
```

```
    int arr[n];
```

```
    printf("Enter %d elements: ", n);
```

```
    for (i = 0; i < n; i++) {
```

```
        scanf("%d", &arr[i]);
```

```
        if (arr[0] < arr[i]) {
```

```
            smallest = arr[i];
```

```
            secsmallest = arr[0];
```

```
        } for (i = 2; i < n; i++) {
```

```
            if (arr[i] < smallest) {
```

```
                secsmallest = smallest;
```

```
                smallest = arr[i];
```

```
            } else if (arr[i] < secsmallest && arr[i] != smallest) {
```

```
                secsmallest = arr[i];
```

```
            } if (smallest == secsmallest) {
```

```
                printf("There is no second smallest element");
```

```
            } else {
```

```
                printf("The second smallest element is: %d",
```

```
                secsmallest);
```

```
            }
```

```
        return 0;
```

```
    }
```


Output:-

Case ① Enter the number of elements in the array: 1
Array must have atleast 2 elements

Case ② Enter the number of array elements: 3
Enter 3 elements: 2
2
2

There is no second smallest element

Case ③ Enter the number of array elements: 4
Enter 4 elements: 3
2
1
4

The second smallest element is 2.

2. To find the sum of the left diagonals of a matrix.

#include <stdio.h>

int main() {

int n, i, j, sum = 0;

printf("Enter the size of square matrix: ");

scanf("%d", &n);

int matrix[n][n];

printf("Enter %d x %d matrix elements: \n", n, n);

for(i=0; i<n; i++) {

for(j=0; j<n; j++) {

scanf("%d", &matrix[i][j]); }

for(i=0; i<n; i++) {

sum += matrix[i][i]; }

printf("Sum of left diagonal elements: %d", sum);

return 0;

}

Output:-

Enter the size of the square matrix: 2

Enter 2x2 matrix elements:-

3

5

2

7

Sum of left diagonal elements: 10.

3. ~~Write~~ C program to simulate cpu scheduling algorithm FCFs

```
#include <stdio.h>
int main()
{
    int n, i;
    printf("Enter number of processes : ");
    scanf("%d", &n);
    int bt[n], wt[n], tat[n];
    for (i=0; i<n; i++)
    {
        printf("Enter burst time for processes %d : ", i+1);
        scanf("%d", &bt[i]);
    }
    wt[0]
```

3. C Program to simulate cpu scheduling algorithm FCFs

```
#include <stdio.h>
int main()
{
    int n, i, j;
    printf("Enter number of processes : ");
    scanf("%d", &n);
    int pid[n], at[n], bt[n], ct[n], wt[n], tat[n], rt[n];
    int temp;
    for (i=0; i<n; i++)
    {
        pid[i] = i+1;
        printf("In Process %d\n", i+1);
        printf("Arrival time : ");
        scanf("%d", &at[i]);
        printf("burst time : ");
        scanf("%d", &bt[i]);
    }
}
```

```
for (i=0; i<n-1; i++)
{
    for (j=i+1; j<n; j++)
    {
        if (at[i] > at[j])
        {
            temp = at[i];
            at[i] = at[j];
            at[j] = temp;

            temp = bt[i];
            bt[i] = bt[j];
            bt[j] = temp;

            temp = pid[i];
            pid[i] = pid[j];
            pid[j] = temp;
        }
    }
}
```

```
int curr_time = 0; int curr_time = 0;
float total_wt = 0, total_wt = 0, total_tat = 0;
for (i=0; i<n; i++)
{
    if (curr_time < at[i])
    {
        curr_time = at[i];
        at[i] = curr_time - at[i];
        curr_time += bt[i];
        ct[i] = curr_time;
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
        total_wt += wt[i];
        total_tat += tat[i];
        total_at += at[i];
    }
}

printf("In PID | AT | BT | CT | TAT | WT | RT |>");
for (i=0; i<n; i++)
{
    printf(" %d | %d | %d | %d | %d | %d | %d",
        pid[i], at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);
}
```



```
printf("In Average Waiting Time = %.2f", total wt / n);
printf("In Average Turnaround Time = %.2f", total tat / n);
printf("In Average Response Time = %.2f", total rt / n);

return 0;
}
```

Output:

Enter number of processes: 4.

Process 1

Arrival time: 0

Burst time: 7

Process 2

Arrival time: 8

Burst time: 3

Process 3

Arrival time: 3

Burst time: 4

Process 4

Arrival time: 5

Burst time: 6

PID	AT	BT	CT	TAT	WT	RT
P1	0	7	7	7	0	0
P3	3	4	11	8	4	4
P4	5	6	17	12	6	6
P2	8	3	20	12	9	9

Average Waiting time = 4.05
Average Turnaround time = 9.05
Average Response time = 4.05

4. SJF - Non-Pre-emptive & Pre-emptive

```
#include <stdio.h>
#include <stdlib.h>

struct process {
    int pid;
    int at;
    int bt;
    int ct;
    int wt, tat, resp, started, completed;
};

struct process *next;
struct process *head = NULL;

void insert(int id, int arrival, int burst) {
    struct process *newNode = (struct process *) malloc(
        sizeof(struct process));
    newNode->pid = id;
    newNode->at = arrival;
    newNode->bt = burst;
    newNode->at = burst;
    newNode->completed = 0;
    newNode->wt = 0;
    newNode->tat = 0;
    newNode->ct = 0;
    newNode->resp = 0;
    newNode->started = 0;
    newNode->next = NULL;
    if (head == NULL)
        head = newNode;
}
```



```

else {
    struct process *temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
        temp->next = new Node;
    }
}

void sort_by_arrival() {
    struct process *p, *q;
    int temp_pid, temp_at, temp_bt, temp_at;
    for (i = head; i != NULL; i = i->next) {
        for (j = p->next; j != NULL; j = j->next) {
            if (i->at > j->at) {
                temp_at = i->at; i->at = j->at; j->at = temp_at;
                temp_bt = i->bt; i->bt = j->bt; j->bt = temp_bt;
                temp_at = i->at; i->at = j->at; j->at = temp_at;
                temp_pid = i->pid; i->pid = j->pid; j->pid = temp_pid;
            }
        }
    }
}

void non_preemptive_sjf(int n) {
    int completed_count = 0;
    int current_time = 0;
    while (completed_count < n) {
        struct process *temp = head;
        struct process *shortest = NULL;
        int min_bt = 1000000;
        while (temp != NULL) {
            if (temp->at <= current_time & temp->completed == 0) {
                if (temp->bt < min_bt) {
                    min_bt = temp->bt;
                    shortest = temp;
                }
            }
            temp = temp->next;
        }
        if (shortest != NULL) {
            temp = shortest;
            temp->at <= current_time;
            temp->bt = min_bt;
            temp->completed = 1;
            completed_count++;
            current_time += min_bt;
        }
    }
}

```

```

temp = temp->next;
}

if (shortest == NULL) {
    temp = head;
    int earliest = 1000000;
    while (temp != NULL) {
        if (temp->completed == 0 & temp->at < earliest) {
            earliest = temp->at;
        }
        temp = temp->next;
    }
    current_time = earliest;
    continue;
}

if (shortest->resp == -1) {
    shortest->resp = current_time - shortest->at;
    if (shortest->resp < 0) shortest->resp = 0;
}

shortest->wt = current_time - shortest->at;
if (shortest->wt < 0) shortest->wt = 0;

current_time = shortest->bt;
shortest->ct = current_time;
shortest->tat = shortest->ct - shortest->at;
shortest->completed = 1;
completed_count++;
}

void preemptive_sjf(int n) {
    int completed = 0;
    int current_time = 0;
    while (completed < n) {
        struct process *temp = head;
        struct process *shortest = NULL;
        int min_at = 1000000;
        while (temp != NULL) {
            if (temp->at <= current_time & temp->completed == 0) {
                if (temp->at < min_at) {
                    min_at = temp->at;
                    shortest = temp;
                }
            }
            temp = temp->next;
        }
        if (shortest != NULL) {
            temp = shortest;
            temp->at <= current_time;
            temp->completed = 1;
            completed++;
            current_time = temp->at;
        }
    }
}

```



```

while (temp != NULL) {
    if (temp->at <= current_time && temp->completed == 0
        && temp->at < min_at) {
        min_at = temp->at;
        shortest = temp;
    }
    temp = temp->next;
}

Switch (shortest != NULL) {
    case 1:
        if (shortest->resp == -1) {
            shortest->resp = current_time - shortest->at;
            if (shortest->resp < 0) shortest->resp = 0;
        }
        shortest->at -= 1;
        current_time++;
        if (shortest->at == 0) {
            shortest->completed = 1;
            shortest->ct = current_time;
            shortest->tat = shortest->ct - shortest->at;
            shortest->wt = shortest->tat - shortest->bt;
            completed++;
        }
        break;
    case 0:
        current_time++;
        break;
}

void display() {
    struct process *temp = head;
    float total_wt = 0, total_tat = 0, total_at = 0;
    int n = 0;

    printf("In Process | AT | BT | CT | TAT | WT | RT |n");

    while (temp != NULL) {
        printf("%d | %d | %d | %d | %d | %d | %d |n",
            temp->pid, temp->at,

```

```

temp->bt, temp->ct, temp->tat, temp->wt, temp->resp);
    total_wt += temp->wt;
    total_tat += temp->tat;
    total_at += temp->at;
    n++;
    temp = temp->next;
}

printf("In Average Waiting Time = %.2f", total_wt/n);
printf("In Average Turnaround Time = %.2f", total_tat/n);
printf("In Average Response Time = %.2f", total_at/n);

int main() {
    int n, i, at, bt, choice;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++) {
        printf("In Process %d |n", i);
        printf("Arrival Time: ");
        scanf("%d", &at);
        printf("Burst Time: ");
        scanf("%d", &bt);
        insert(i, at, bt);
    }

    Sort-by-arrival();

    printf("In Select Scheduling Type: |n");
    printf("1. Non-preemptive SJF |n");
    printf("2. Preemptive SJF |n");
    printf("Enter Choice: ");
    scanf("%d", &choice);
    switch (choice) {
        case 1:
            non-preemptive_sjf(n);
            break;
        case 2:
            preemptive_sjf(n);

```



```

break;
default:
    printf("Invalid choice!\n");
    return 1;
}
display();
return 0;
}

```

Output :

Enter number of processes: 4.

Process P1

Arrival Time: 0

Burst Time: 4

Process P2

Arrival Time: 8

Burst Time: 3

Process P3

Arrival Time: 3

Burst Time: 4

Process P4

Arrival Time: 5

Burst Time: 6

Select Scheduling Type:

1. Non-Preemptive SJF

2. Preemptive SJF

Enter Choice: 1

Process	AT	BT	CT	TAT	WT	RT
P1	0	4	4	4	0	0
P3	3	4	11	8	4	4
P4	5	6	20	15	9	9
P2	8	3	14	6	3	3

Average Waiting Time = 4.00

Average Turnaround Time = 9.00

Average Response Time = 4.00

Enter choice: 2.

Process	AT	BT	CT	TAT	WT	RT
P1	0	4	4	4	0	0
P3	3	4	11	8	4	4
P4	5	6	20	15	9	9
P2	8	3	14	6	3	3

Average Waiting Time = 4.00

Average Turnaround Time = 9.00

Average Response Time = 4.00

Observation:-

On comparing the average times of FCFS & SJF it can be concluded that the average times for SJF are lower than that of FCFS. Therefore, SJF is more optimal.

WAP to Implement Priority (Pre-emptive, non-preemptive) & Round Robin.

```
#include <stdio.h>
#define MAX 20

void priority_np(int n, int at[], int bt[], int pr[])
{
    int wt[MAX] = {0}, tat[MAX], rt[MAX], ct[MAX], done[MAX] = {0};
    int time = 0, count = 0;
    float sw = 0, st = 0, sr = 0;
    while (count < n)
    {
        int idx = -1, min = 9999;
        for (int p = 0; p < n; p++)
        {
            if (at[p] <= time && !done[p] && pr[p] < min)
            {
                min = pr[p];
                idx = p;
            }
            if (idx != -1)
            {
                at[idx] = time - at[idx];
                wt[idx] = time - at[idx];
                time += bt[idx];
                ct[idx] = time;
                tat[idx] = ct[idx] - at[idx];
                done[idx] = 1;
                count++;
            }
            else
            {
                time++;
            }
        }
    }
}
```

```
printf("In PID | AT | BT | CT | WT | TAT | RT | \n");
for (int p = 0; p < n; p++)
{
    printf("%d | %d | %d | %d | %d | %d | %d | \n",
        p+1, at[p], bt[p], ct[p], wt[p], tat[p], rt[p]);
}
```

```
swt = wt[p];
st += tat[p];
sat = at[p];
printf("In Avg WT = %.2f | In Avg TAT = %.2f | In Avg RT = %.2f | \n",
    sw/n, st/n, sat/n);
}

void priority_p(int n, int at[], int bt[], int pr[])
{
    int rt[MAX], wt[MAX] = {0}, tat[MAX], at[MAX], ct[MAX];
    for (int p = 0; p < n; p++)
    {
        at = bt[p] = bt[p];
        at[p] = -1;
    }
    int time = 0, complete = 0;
    float sw = 0, st = 0, sr = 0;
    while (complete < n)
    {
        int idx = -1, min = 9999;
        for (int p = 0; p < n; p++)
        {
            if (at[p] <= time && at - bt[p] > 0 && pr[p] < min)
            {
                min = pr[p];
                idx = p;
            }
            if (idx != -1)
            {
                if (at[idx] == -1)
                {
                    at[idx] = time - at[idx];
                    bt[idx] = time - at[idx] + time - at[idx];
                }
                else
                {
                    time++;
                }
            }
        }
        printf("In PID | AT | BT | CT | WT | TAT | RT | \n");
        for (int p = 0; p < n; p++)
        {
            printf("%d | %d | %d | %d | %d | %d | %d | \n",
                p+1, at[p], bt[p], ct[p], wt[p], tat[p], rt[p]);
            swt += wt[p];
            st += tat[p];
            sr += at[p];
        }
    }
}
```



```

void aa(int n, int at[], int bt[], int q) {
    int st[100], wt[100] = {0}, tat[100], at[100],
    ct[100];
    int done[100] = {0};
    for (int i = 0; i < n; i++) {
        at[i] = bt[i];
        ct[i] = -1;
    }
    int time = 0, complete = 0;
    float sw = 0, st = 0, sr = 0;
    float avgsw, avgst, avgsr;
    while (complete < n) {
        for (int p = 0; p < n; p++) {
            if (at[p] < time & at[p] > 0) {
                if (at[p] == -1)
                    at[p] = q;
            }
            else {
                time = at[p];
            }
            ct[p] = time;
            at[p] = 0;
            tat[p] = ct[p] - at[p];
            wt[p] = tat[p] - bt[p];
            complete++;
        }
        time++;
        printf("\n PID | AT | BT | CT | WT | TAT | RT | n");
        for (int i = 0; i < n; i++) {
            printf("%d | %d | %d | %d | %d | %d | %d |",
                i+1, at[i], bt[i], ct[i], wt[i], tat[i], at[i]);
            sw += wt[i];
            st += tat[i];
            sr += at[i];
        }
        printf("\n Avg WT = %.2f | Avg TAT = %.2f | Avg RT = %.2f",
            sw/n, st/n, sr/n);
    }
}

```

```

int main() {
    int n, choice, q;
    int at[100], bt[100], pr[100];
    printf("Processes : ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("AT BT PR for P %d : ", i+1);
        scanf("%d %d %d", &at[i], &bt[i], &pr[i]);
    }
    printf("\n 1. Priority Non-Preemptive");
    printf("\n 2. Priority Preemptive");
    printf("\n 3. Round Robin");
    printf("\n Choice : ");
    scanf("%d", &choice);
    switch (choice) {
        case 1:
            priority_npr(at, bt, pr);
            break;
        case 2:
            priority_p(n, at, bt, pr);
            break;
        case 3:
            printf("Quantum : ");
            scanf("%d", &q);
            aa(n, at, bt, q);
            break;
    }
    return 0;
}

```


O/P:-

Enter number of processes: 5
Enter AT BT PR for P1: 0 3 5
Enter AT BT PR for P2: 2 2 3
Enter AT BT PR for P3: 3 5 2
Enter AT BT PR for P4: 4 4 4
Enter AT BT PR for P5: 6 1 1

1. Priority non Preemptive
 2. Priority Preemptive
 3. Round Robin.
- Enter Choice: 1

Gantt chart:

	P1	P3	P5	P2	P4	
0	3	8	9	11	15	
PID	AT	BT	CT	WT	TAT	RT
1	0	3	3	0	3	0
2	2	2	11	7	9	7
3	3	5	8	0	5	0
4	4	4	15	7	11	7
5	6	1	9	2	3	2

Avg WT: 3.20

Avg TAT: 6.20

Avg RT: 3.20

Enter Choice: 2

Gantt chart:

P1	P2	P3	P5	P3	P2	P4	P1
0	2	3	6	7	9	10	15

Enter Choice: 3.
Enter Time Quantum: 4.

Gantt chart:

P1	P2	P3	P4	P5	P3
0	3	5	9	13	16

PID	AT	BT	CT	WT	TAT	RT
1	0	3	3	0	3	0
2	2	3	5	1	1	1
3	3	5	16	8	13	2
4	4	4	13	5	9	5
5	6	1	14	7	8	7

Avg WT = 4.20

Avg TAT = 7.20

Avg RT = 3.00

Enter Choice: 2

Gantt chart:

P1	P2	P3	P5	P3	P2	P4	P1
0	2	3	6	7	9	10	15

PID	AT	BT	CT	WT	TAT	RT
1	0	3	15	12	15	0
2	2	2	10	6	8	0
3	3	5	9	1	6	0
4	4	4	14	6	10	6
5	6	1	7	0	1	0

Avg WT = 5.00

Avg TAT = 8.00

Avg RT = 1.20

Multi-level Scheduling

```
#define <stdio.h>
#define <string.h>
#define MAX 100

typedef struct {
    int pid;
    int arrival;
    int burst;
    int remaining;
    int completion;
    int waiting;
    int turnaround;
    char type[10];
} Process;

typedef struct {
    int items[MAX];
    int front, rear;
} Queue;

void initQueue(Queue *q) {
    q->front = q->rear = -1;
}

void isEmpty(Queue *q) {
    return (q->front == -1);
}

void enqueue(Queue *q, int val) {
    if (q->rear == MAX) return;
    if (q->front == -1) q->front = 0;
    q->items[++q->rear] = val;
}

int dequeue(Queue *q) {
    if (isEmpty(q)) return -1;
    int val = q->items[q->front];
    if (q->front == q->rear)
```

```
q->front = q->rear = -1;
else
    q->front++;
return val;
}

void SortbyArrival(Process p[], int n) {
    Process temp;
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (p[i].arrival > p[j].arrival) {
                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }
}

int main() {
    int n;
    Process p[MAX];
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("In Process %d\n", i + 1);
        printf("PID: ");
        scanf("%d", &p[i].pid);
        printf("Arrival Time: ");
        scanf("%d", &p[i].arrival);
        printf("Burst Time: ");
        scanf("%d", &p[i].burst);
        printf("Type [system/user]: ");
        scanf("%s", p[i].type);
        p[i].remaining = p[i].burst;
    }
    SortbyArrival(p, n);
    Queue system, user;
```



```

initQueue(&systemQ);
initQueue(&userQ);
int time = 0, completed = 0, i = 0;
int current = -1;
int qant[100], gtime[100];
int q_index = 0;
while (completed < n) {
    while (i < n && p[i].arrival <= time) {
        if (strcmp(p[current].type, "user") == 0 &&
            !isEmpty(&systemQ)) {
            enqueue(&userQ, current);
            current = -1;
        }
    }
    if (current == -1) {
        if (!isEmpty(&systemQ))
            current = dequeue(&systemQ);
        else if (!isEmpty(&userQ))
            current = dequeue(&userQ);
        else {
            time++;
            continue;
        }
    }
    gant[q_index] = p[current].pid;
    gtime[q_index] = time;
    q_index++;
    p[current].remaining--;
    time++;
    if (p[current].remaining == 0) {
        p[current].completion = time;
        completed++;
        current = -1;
    }
}

```

```

gtime[q_index] = time;
float totalWT = 0, totalTAT = 0;
for (int i = 0; i < n; i++) {
    p[i].turnaround = p[i].completion - p[i].arrival;
    p[i].waiting = p[i].turnaround - p[i].burst;
    totalWT += p[i].waiting;
    totalTAT += p[i].turnaround;
}

printf("\n\n Gantt Chart: \n\n");
for (int i = 0; i < q_index; i++) {
    printf("P%d ", gant[i]);
}
printf("\n\n");
for (int i = 0; i < q_index; i++) {
    printf("P%d ", gtime[i]);
}

printf("\n\n PID\tType\tAT\tBT\tCT\tTAT\tWT\n");
for (int i = 0; i < n; i++) {
    printf("P%d\t%s\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid,
        p[i].type,
        p[i].arrival,
        p[i].burst,
        p[i].completion,
        p[i].turnaround,
        p[i].waiting);
}

printf("\n Average Waiting Time = %.2f", totalWT/n);
printf("\n Average Turnaround Time = %.2f", totalTAT/n);

return 0;
}

```


Enter number of processes: 4

Process 1

PID: 0

Arrival Time: 0

Burst Time: 5

Type (system/user): system

Process 2

PID: 1

Arrival Time: 2

Burst Time: 3

Type (system/user): user

Process 3

PID: 2

Arrival Time: 1

Burst Time: 4

Type (system/user): ~~user~~ system

Process 4

PID: 3

Arrival Time: 3

Burst Time: 2

Type (system/user): user

Gantt Chart:

P0	P2	P1	P3
0	5	9	12

PID	Type	AT	BT	CT	TAT	WT
P0	system	0	5	5	5	0
P2	system	1	4	9	8	4
P1	user	2	3	12	10	7
P3	user	3	2	14	11	9

Average waiting Time = 5.00

Average Turnaround Time = 8.50

10/10


```
EDF & RMG:
#include <stdio.h>
#define MAX 10
typedef struct {
    int id;
    int execution;
    int period;
    int deadline;
    int remaining;
    int completion;
} Task;

void edge1 Task t[], int n) {
    printf("In ===== Earliest Deadline First (EDF) Scheduling\n");
printf("ID | ET | BT | Deadline | TTT | TT = == ln")
    printf("ID | ET | BT | Deadline | TTT | TT = == ln");
    int time = 0, completed = 0;
    int deadlines[MAX];
    for (int i = 0; i < n; i++) {
        t[i].remaining = t[i].execution;
        deadlines[i] = t[i].deadline;
    }
    while (completed < n) {
        int earliest = -1;
        for (int i = 0; i < n; i++) {
            if (t[i].remaining > 0 ^ deadlines[i] <= earliest) {
                earliest = deadlines[i];
                selected = i;
            }
        }
        if (selected != -1) {
            t[selected].remaining--;
            if (t[selected].remaining == 0) {
                t[selected].completion = time + 1;
            }
        }
    }
}
```

```

    }
    time++;
}
for (int i=0; i<n; i++) {
    int tat = t[i].completion;
    int wt = tat - t[i].execution;
    printf("%d %d %d %d %d %d %d %d %d\n",
        t[i].id, t[i].execution, t[i].deadline,
        t[i].completion, wt, tat);
}
}

void run(Task t[], int n) {
    printf("\n ===== Rate Monotonic Scheduling (RMS) =====\n");
    printf("ID | T | Period | t | T | two | t | TAT | n\n");
    int time = 0, completed = 0;
    for (int i=0; i<n; i++) {
        t[i].remaining = t[i].execution;
        while (completed < n) {
            int min_period = 9999, selected = -1;
            for (int i=0; i<n; i++) {
                if (t[i].remaining > 0 && t[i].period < min_period) {
                    min_period = t[i].period;
                    selected = i;
                }
            }
            if (selected != -1) {
                t[selected].remaining--;
                if (t[selected].remaining == 0) {
                    t[selected].completion = time + 1;
                    completed++;
                }
                time++;
            }
        }
    }
}

```



```
for(int i=0; i<n; i++) {
    int tat = t[i].completion;
    int wt = tat - t[i].execution;
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\t",
        t[i].id, t[i].execution, t[i].period, t[i].completion,
        wt, tat);
}
```

```
// void proportional (Task t[], int n) {
    printf("n == Proportional Share Scheduling ==\n");
    printf("ID\tET\tShare\tCT\tWT\tTAT\n");
    int total = 0;
    for(int i=0; i<n; i++) {
        total += t[i].execution;
    }
    int time = 0;
    for(int i=0; i<n; i++) {
        t[i].completion = time + t[i].execution;
        time += t[i].execution;
    }
    for(int i=0; i<n; i++) {
        int tat = t[i].completion;
        int wt = tat - t[i].execution;
        int share = (t[i].execution * 100) / total;
        printf("%d\t%d\t%d\t%d\t%d\t%d\t",
            t[i].id, t[i].execution, share, t[i].
            completion, wt, tat);
    }
}
```

```
int main() {
    int n;
    Task t[MAX];
    printf("Enter number of tasks: ");
    scanf("%d", &n);
```

```
for(int i=0; i<n; i++) {
    printf("Task %d\n", i+1);
    t[i].id = i+1;
    printf("Execution Time: ");
    scanf("%d", &t[i].execution);
    printf("Period: ");
    scanf("%d", &t[i].period);
```

```
printf("Deadline: ");
scanf("%d", &t[i].deadline);
}
}
// void proportional (t, n) {
    return 0;
}
```

```
Output:
Enter number of tasks: 3
Task 1
Execution Time: 3
Period: 6
Deadline: 10
Task 2
Execution Time: 2
Period: 4
Deadline: 5
```


Task 3

Execution Time: 1, 2, 3

Period: 3

Deadline: 8

==== Earliest Deadline First (EDF) Scheduling =====

ID	BT	Deadline	CT	WT	TAT
1	3	10	6	3	6
2	2	5	2	0	2
3	1	8	3	2	3

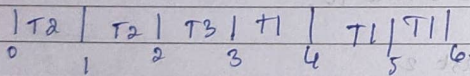
==== Rate Monotonic Scheduling (RMS) =====

ID	BT	period	CT	WT	TAT
1	3	6	6	3	6
2	2	4	3	1	3
3	1	3	1	0	1

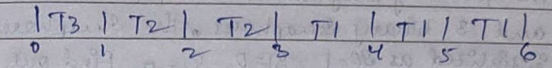
==== Proportional Share Scheduling =====

ID	BT	Share	CT	WT	TAT
1	3	50	3	0	3
2	2	33	5	3	5
3	1	16	6	5	6

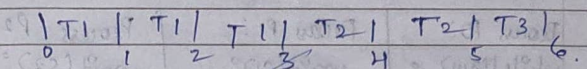
Gantt Chart (EDF):



Gantt Chart: (RMS)



Gantt Chart: (Proportional)



8. The following program consists of 3 concurrent processes and 3 binary semaphores. The semaphores are initialized as $S_0=1$, $S_1=0$, $S_2=0$. Find out how many times process will print 'D'. Assume order of execution as P_0, P_1, P_2, P_0, P_1 .

Process P_0 :	Process P_1 :	Process P_2 :
while(true) {	wait(S_1);	wait(S_2);
wait(S_0);	signal(S_0);	signal(S_0);
Print("D");		
signal(S_1);		
signal(S_2);		
}		

Ans	Initial	$S_0=1$	$S_1=0$	$S_2=0$	Remark
	P_0	0	1	1	0
	P_1	1	0	1	-
	P_2	1	0	0	-
	P_0	0	1	1	DD
	P_1	0	1	0	-

P_0 will print "D" two times. wait $\rightarrow$ decrease
signal $\rightarrow$ increase

Producer-Consumer.

O/P :-
Enter the number of items: 3
Produced: 0 at buffer [0]
Produced: 1 at buffer [1]
consumed: 0 at buffer [0]
Produced: 2 at buffer [2]
consumed: 1 at buffer [1]
~~Produced: 3 at buffer [3]~~
consumed: 2 at buffer [2]

Code :-

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#define N 5
int buffer[N], in = 0, out = 0, item = 0;
sem_t full, mutex;

void* producer(void* arg) {
    for(int i = 0; i < N; i++) {
        sem_wait(&full);
        sem_wait(&mutex);
        buffer[in] = item;
        printf("Produced %d at buffer [%d]\n", item, in);
        in = (in + 1) % N;
        item++;
        sem_post(&mutex);
        sem_post(&full);
    }
    return NULL;
}

void* consumer(void* arg) {
    for(int i = 0; i < N; i++) {
        sem_wait(&full);
        sem_wait(&mutex);
```



```

int Val = buffer[out];
printf("consumed: %d from buffer [%d] in ", Val, Out);
out = (out + 1) % N;
sem_post(&mutex);
sem_post(&empty);
return NULL;
}

int main() {
pthread_t p, c;
printf("Enter number of items: ");
scanf("%d", &n);
sem_init(&empty, 0, N);
sem_init(&full, 0, 0);
sem_init(&mutex, 0, 1);
pthread_create(&p, NULL, producer, NULL);
pthread_create(&c, NULL, consumer, NULL);
pthread_join(p, NULL);
pthread_join(c, NULL);
return 0;
}

```

Dining Philosopher

```

#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <stdlib.h>

int N;
sem_t *forks, mutex;
void* philosopher(void* num) {
int P = *(int*)num;
for (int i = 0; i < 2; i++) {
printf("Philosopher %d is thinking.\n", P);
sem_wait(&mutex);
sem_wait(&forks[P]);
printf("Philosopher %d picked up left fork %d.\n", P, P);
sem_wait(&forks[(P+1)%N]);
printf("Philosopher %d picked up right fork %d.\n", P, (P+1)%N);
sem_post(&mutex);
printf("Philosopher %d is eating.\n", P);
sem_post(&forks[P]);
sem_post(&forks[(P+1)%N]);
printf("Philosopher %d put down forks %d and %d.\n", P, P, (P+1)%N);
}
return NULL;
}

int main() {
printf("Enter number of philosophers: ");
scanf("%d", &N);
pthread_t p[N];
int id[N];
forks = (sem_t*) malloc(N * sizeof(sem_t));
sem_init(&mutex, 0, 1);

```



```
for (int i=0; i<N; i++) {
    sem_wait(&forks[i], 0, 1);
    for (int i=0; i<N; i++) {
        id[i] = i;
        pthread_create(&p[i], NULL, philosopher,
            &id[i]);
    }
    for (int i=0; i<N; i++) {
        pthread_join(p[i], NULL);
    }
}
```

return 0;

O/p :-

enter number of philosopher: 4.

Philosopher 1 is thinking
 Philosopher 1 picked up left fork 1
 Philosopher 1 picked up right fork 2
 Philosopher 3 is thinking
 Philosopher 3 picked up left fork 3
 Philosopher 0 is thinking
 Philosopher 3 picked up right fork 0
 Philosopher 1 is eating
 Philosopher 3 is eating
 Philosopher 1 put down forks 1 and 2
 Philosopher 0 picked up left fork 0.
 Philosopher 1 is thinking
 Philosopher 0 picked up right fork 1
 Philosopher 0 is eating
 Philosopher 0 put down forks 0 and 1
 Philosopher 0 is thinking
 Philosopher 1 picked up left fork 1
 Philosopher 1 picked up right fork 2
 Philosopher 2 is thinking

Philosopher 3 put down forks 3 and 0
 Philosopher 3 is thinking
 Philosopher 0 picked up left fork 0.
 Philosopher 1 is eating
 Philosopher 1 put down forks 1 and 2.
 Philosopher 0 picked up right fork 1.
 Philosopher 0 is eating
 Philosopher 0 put down forks 0 and 1
 Philosopher 2 picked up left fork 2.
 Philosopher 2 picked up right fork 3
 Philosopher 2 is eating
 Philosopher 2 put down forks 2 and 3.
 Philosopher 2 is thinking
 Philosopher 3 picked up left fork 3.
 Philosopher 3 picked up right fork 0.
 Philosopher 3 is eating
 Philosopher 3 put down forks 3 and 0.
 Philosopher 2 picked up left fork 2.
 Philosopher 2 picked up right fork 3
 Philosopher 2 is eating
 Philosopher 2 put down forks 2 and 3.

21/11/26

Safety Algorithm :-

```
#include <stdio.h>
#include <stdbool.h>
int main()
{
    int P, R;
    int i, j;
    printf("Enter number of processes : ");
    scanf("%d", &P);
    printf("Enter number of resource types : ");
    scanf("%d", &R);
    int Allocation[P][R], Max[P][R], Need[P][R];
    int Available[R], Work[R];
    bool Finish[P];
    int SafeSequence[P];
    printf("Enter Allocation Matrix: \n");
    for (i = 0; i < P; i++)
    {
        for (j = 0; j < R; j++)
        {
            scanf("%d", &Allocation[i][j]);
        }
    }
    printf("Enter Max Matrix: \n");
    for (i = 0; i < P; i++)
    {
        for (j = 0; j < R; j++)
        {
            scanf("%d", &Max[i][j]);
        }
    }
    printf("Enter Available Resources: \n");
    for (j = 0; j < R; j++)
    {
        scanf("%d", &Available[j]);
    }
    for (i = 0; i < P; i++)
    {
        for (j = 0; j < R; j++)
        {
            Need[i][j] = Max[i][j] - Allocation[i][j];
        }
    }
}
```

```
for (j = 0; j < R; j++)
{
    Work[j] = Available[j];
}
for (i = 0; i < P; i++)
{
    Finish[i] = false;
}
int count = 0;
while (count < P)
{
    bool found = false;
    for (i = 0; i < P; i++)
    {
        if (Finish[i] == false)
        {
            bool canExecute = true;
            for (j = 0; j < R; j++)
            {
                if (Need[i][j] > Work[j])
                {
                    canExecute = false;
                    break;
                }
            }
            if (canExecute)
            {
                for (j = 0; j < R; j++)
                {
                    Work[j] += Allocation[i][j];
                }
                Finish[i] = true;
                SafeSequence[count++] = i;
                found = true;
            }
            if (found == false)
            {
                break;
            }
        }
        bool safe = true;
        for (i = 0; i < P; i++)
        {
            if (Finish[i] == false)
            {
                safe = false;
                break;
            }
        }
        if (safe)
        {
            printf("System is in SAFE STATE in Safe Sequence : ");
            for (i = 0; i < P; i++)
            {
                printf("%d ", SafeSequence[i]);
            }
        }
    }
}
```



```
printf("%d", SafeSequence[PJ]);  
printf("\n");  
else {  
    printf("In System is in UNSAFE STATE  
    Potential Deadlock\n");  
}  
return 0;
```

DIP:

Enter number of Processes : 5
Enter number of Resource types : 4
Enter Allocation matrix :

0	0	1	2
1	0	0	0
1	3	5	4
0	6	3	2
0	0	1	4

Enter Max Matrix :

1	7	5	0
2	3	5	6
0	6	5	2
0	6	5	6

Enter Available Resources :

1	5	2	0
---	---	---	---

System is in Safe state

Safe Sequence: P0 P2 P3 P4 P1

Banker's Algorithm:

```
#include <stdio.h>  
#include <stdbool.h>  
int main() {  
    int P, R, i, j;  
    printf("Enter number of processes : ");  
    scanf("%d", &P);  
    printf("Enter number of resource types : ");  
    scanf("%d", &R);  
    int Allocation[P][R], Max[P][R], Need[P][R];  
    int Available[R], Work[R];  
    bool Finish[P];  
    int SafeSequence[P];  
    int Request[R], process;  
    printf("Enter Allocation matrix :\n");  
    for (i=0; i<P; i++) {  
        for (j=0; j<R; j++) {  
            scanf("%d", &Allocation[i][j]);  
        }  
    }  
    printf("Enter Max matrix :\n");  
    for (i=0; i<P; i++) {  
        for (j=0; j<R; j++) {  
            scanf("%d", &Max[i][j]);  
        }  
    }  
    printf("Enter Available Resources :");  
    for (j=0; j<R; j++) {  
        scanf("%d", &Available[j]);  
    }  
    for (i=0; i<P; i++) {  
        for (j=0; j<R; j++) {  
            Need[i][j] = Max[i][j] - Allocation[i][j];  
        }  
    }  
    printf("Enter process number (0 to %d) : ", P-1);  
    scanf("%d", &process);  
    printf("Enter request vector :\n");  
    for (j=0; j<R; j++) {  
        scanf("%d", &Request[j]);  
    }  
}
```



```

for (j=0; j<R; j++) {
    if (Request[P] > Need[process][P]) {
        printf("In fages: Request exceeds Need!\n");
        return 0;
    }
    if (Request[P] > Available[P]) {
        printf("Resources not available. Process must wait.\n");
        return 0;
    }
    for (i=0; i<P; i++) {
        Available[P] -= Request[P];
        Allocation[process][P] += Request[P];
        Need[process][P] -= Request[P];
    }
    for (j=0; j<R; j++) {
        work[j] = Available[j];
    }
    for (i=0; i<P; i++) {
        Finish[i] = false;
    }
    int count = 0;
    while (count < P) {
        bool found = false;
        for (p=0; p<P; p++) {
            if (!Finish[p]) {
                bool canExecute = true;
                for (j=0; j<R; j++) {
                    if (work[j] < Need[p][j]) {
                        canExecute = false;
                        break;
                    }
                }
                if (canExecute) {
                    for (j=0; j<R; j++) {
                        work[j] += Allocation[p][j];
                    }
                    Finish[p] = true;
                    safeSequence[count++] = p;
                }
            }
        }
    }
}

```

```

found = true; } } }
if (!found)
    break;
}
if (count == P) {
    printf("In REQUEST GRANTED. System is in SAFE STATE\n Sequence:");
    for (p=0; p<P; p++) {
        printf(" %d ", safeSequence[p]);
    }
    else {
        printf("In REQUEST DENIED. System would be UNSAFE\n");
    }
    return 0;
}

```

Output:

Enter number of processes: 5
Enter number of resource types: 3

Enter Allocation matrix:

0	1	0
2	0	0
3	0	2
2	1	1
0	0	2

Enter Max Matrix:

4	5	3
3	2	2
9	0	2
2	2	2
4	3	3

5/5/26

Enter Available Resources: 3 3 2
Enter process number (0 to 4): 1
Enter request vector: 1 0 2
Request GRANTED. system is in SAFE STATE.
Sequence: P1 P3 P4 P0 P2.

Deadlock Detection

```
#include <stdio.h>
#include <stdbool.h>

int main() {
    int P, R, i, j;

    printf("Enter number of processes: ");
    scanf("%d", &P);

    printf("Enter number of resource types: ");
    scanf("%d", &R);

    int Allocation[P][R], Request[P][R];
    int Available[R], Work[R];
    bool Finish[P];

    printf("Enter Allocation Matrix: ");
    for (i = 0; i < P; i++) {
        for (j = 0; j < R; j++) {
            scanf("%d", &Allocation[i][j]);
        }
    }

    printf("Enter Request Matrix: ");
    for (i = 0; i < P; i++) {
        for (j = 0; j < R; j++) {
            scanf("%d", &Request[i][j]);
        }
    }

    printf("Enter Available Resources: ");
    for (j = 0; j < R; j++) {
        scanf("%d", &Available[j]);
    }

    Work[j] = Available[j];

    for (i = 0; i < P; i++) {
        Finish[i] = false;
        for (j = 0; j < R; j++) {
            if (Allocation[i][j] != 0) {
                Finish[i] = false;
                break;
            }
        }
    }

    bool found;
```



```

do {
    found = false;
    for (i = 0; i < P; i++) {
        if (!Finish[i]) {
            bool can_execute = true;
            for (j = 0; j < R; j++) {
                if (Request[i][j] > work[j]) {
                    can_execute = false;
                    break;
                }
            }
            if (can_execute) {
                for (j = 0; j < R; j++) {
                    work[j] += Allocation[i][j];
                }
                Finish[i] = true;
                found = true;
            }
        }
    } while (!found);

    bool deadlock = false;
    for (i = 0; i < P; i++) {
        if (!Finish[i]) {
            deadlock = true;
            printf("Process P%d is\n", i);
            printf("deadlocked\n");
        }
    }
    if (!deadlock) {
        printf("No Deadlock Detected\n");
        return 0;
    }
}

```

O/P:

Enter number of processes: 3
 Enter number of resource types: 2
 Enter Allocation matrix:

0	1
1	1
0	0

Enter Request matrix:

1	1
1	0
0	1

Enter Available Resources:

0	0
---	---

Process P0 is deadlocked
 Process P1 is deadlocked
 Process P2 is deadlocked.

Memory allocation:- best, worst, first fit

```
#include <stdio.h>
#define MAX 100
void firstFit(int blocks[], int m, int processes[], int n) {
    int allocation[MAX];
    for (int i = 0; i < n; i++)
        allocation[i] = -1;
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            if (blocks[j] >= processes[i]) {
                allocation[i] = j;
                blocks[j] -= processes[i];
                break;
            }
        }
        printf("In FIRST FIT ALLOCATION\n");
        printf("Processes Size < Block\n");
        for (int i = 0; i < n; i++) {
            printf("Process %d\n", i, processes[i]);
            if (allocation[i] != -1)
                printf("%d\n", allocation[i] + 1);
            else
                printf("Not Allocated\n");
        }
    }
}

void bestFit(int blocks[], int m, int processes[], int n) {
    int allocation[MAX];
    for (int i = 0; i < n; i++)
        allocation[i] = -1;
    for (int i = 0; i < m; i++) {
        int bestIdx = -1;
        for (int j = 0; j < n; j++) {
            if (blocks[j] >= processes[i]) {
                if (bestIdx == -1 || blocks[j] < blocks[bestIdx])
                    bestIdx = j;
            }
        }
        if (bestIdx != -1) {
            allocation[i] = bestIdx;
            blocks[bestIdx] -= processes[i];
        }
    }
}
```

```
if (bestIdx != -1) {
    allocation[i] = bestIdx;
    blocks[bestIdx] -= processes[i];
}
printf("In BEST FIT ALLOCATION\n");
printf("Processes Size < Block\n");
for (int i = 0; i < n; i++) {
    printf("Process %d\n", i, processes[i]);
    if (allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}

void worstFit(int blocks[], int m, int processes[], int n) {
    int allocation[MAX];
    for (int i = 0; i < n; i++)
        allocation[i] = -1;
    for (int i = 0; i < m; i++) {
        int worstIdx = -1;
        for (int j = 0; j < n; j++) {
            if (blocks[j] >= processes[i]) {
                if (worstIdx == -1 || blocks[j] > blocks[worstIdx])
                    worstIdx = j;
            }
        }
        if (worstIdx != -1) {
            allocation[i] = worstIdx;
            blocks[worstIdx] -= processes[i];
        }
    }
    printf("Worst FIT ALLOCATION\n");
    printf("Processes Size < Block\n");
    for (int i = 0; i < n; i++) {
        printf("Process %d\n", i, processes[i]);
        if (allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}
```



```

printf("%d\n", allocation[P]+1);
else
    printf("Not Allocated\n");
}
int main()
{
    int m, n, choice;
    int blocks[MAX], processes[MAX];
    int tempBlocks[MAX];
    printf("Enter number of memory blocks: ");
    scanf("%d", &m);
    printf("Enter sizes of memory blocks\n");
    for (int i=0; i<m; i++)
        scanf("%d", &blocks[i]);
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter sizes of processes\n");
    for (int i=0; i<n; i++)
        scanf("%d", &processes[i]);
    do {
        printf("\n--- MEMORY ALLOCATION MENU ---\n");
        printf("1. First Fit\n");
        printf("2. Best Fit\n");
        printf("3. Worst Fit\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        for (int i=0; i<n; i++)
            tempBlocks[i] = blocks[i];
        switch (choice) {
            case 1:
                firstFit(tempBlocks, m, processes, n);
                break;
            case 2:
                bestFit(tempBlocks, m, processes, n);
                break;
            case 3:
                worstFit(tempBlocks, m, processes, n);
                break;
            case 4:
                printf("Exiting Program...\n");
                break;
            default:
                printf("Invalid Choice!\n");
        }
    } while (choice != 4);
    return 0;
}

```

```

bestFit(tempBlocks, m, processes, n);
break;
}
case 3:
    worstFit(tempBlocks, m, processes, n);
    break;
case 4:
    printf("Exiting Program...\n");
    break;
default:
    printf("Invalid Choice!\n");
}
} while (choice != 4);
return 0;
}

```

Op:-
 Enter number of memory blocks: 6
 Enter sizes of memory blocks:
 300
 600
 350
 200
 150
 125
 Enter number of processes: 5
 Enter sizes of processes:
 115
 500
 358
 200
 315

--- MEMORY ALLOCATION MENU ---

1. First FIT

2. Best FIT

3. Worst FIT

4. Exit

Enter your choice: 1

FIRST FIT ALLOCATION

Process	Size	Block
P1	115	1
P2	500	2
P3	358	5
P4	200	3
P5	345	Not Allocated.

Enter your choice: 2.

BEST FIT ALLOCATION

Process	Size	Block
P1	115	6
P2	500	2
P3	358	5
P4	200	4
P5	345	Not Allowed.

Enter your choice: 3

WORST FIT ALLOCATION

Process	Size	Block
P1	115	5
P2	500	5
P3	358	2
P4	200	3
P5	345	Not Allocated.

Enter your choice: 4

Exiting Program...

Replacement Algorithm

```
#include <stdio.h>
```

```
#define MAX 50
```

```
int isPresent(int frames[], int n, int page) {
```

```
for (int i = 0; i < n; i++) {
```

```
if (frames[i] == page)
```

```
return 1;
```

```
} return 0;
```

```
}
```

```
void FIFO (int pages[], int n, int capacity) {
```

```
int frames[MAX], index = 0, faults = 0;
```

```
for (int i = 0; i < n; i++) {
```

```
frames[i] = -1;
```

```
printf("In FIFO Page Replacement\n");
```

```
for (int i = 0; i < n; i++) {
```

```
if (!isPresent(frames, capacity, pages[i])) {
```

```
frames[index] = pages[i];
```

```
index = (index + 1) % capacity;
```

```
faults++;
```

```
printf("Page %d -> ", pages[i]);
```

```
for (int j = 0; j < capacity; j++) {
```

```
if (frames[j] == -1)
```

```
printf("%d", frames[j]);
```

```
else
```

```
printf(" ");
```

```
}
```

```
printf("\n");
```

```
} printf("Total Page Faults = %d\n", faults);
```

```
} void LRU (int pages[], int n, int capacity) {
```

```
int frames[MAX], time[MAX];
```

```
int faults = 0, counter = 0;
```



```

for (int i=0; i<capacity; i++) {
    frames[i] = -1;
    time[i] = 0;
}
printf("In LRU Page Replacement\n");
for (int i=0; i<n; i++) {
    int found = 0;
    for (int j=0; j<capacity; j++) {
        if (frames[j] == pages[i]) {
            counter++;
            time[j] = counter;
            found = 1;
            break;
        }
    }
    if (!found) {
        int pos = 0;
        for (int j=1; j<capacity; j++) {
            if (time[j] < time[pos]) {
                pos = j;
            }
        }
        counter++;
        frames[pos] = pages[i];
        time[pos] = counter;
        faults++;
    }
    printf("Page %d -> ", pages[i]);
    for (int j=0; j<capacity; j++) {
        if (frames[j] != -1) {
            printf("%d ", frames[j]);
        }
        else {
            printf("- ");
        }
    }
    printf("\n");
    printf("Total Page Faults = %d\n", faults);
}

```

```

void optimal(int pages[], int n, int capacity) {
    int frames[MAX];
    int faults = 0;
    for (int i=0; i<capacity; i++) {
        frames[i] = -1;
    }
    printf("In Optimal Page Replacement\n");
    for (int i=0; i<n; i++) {
        if (!isPresent(frames, capacity, pages[i])) {
            int pos = 1, farthest = i+1;
            for (int j=0; j<capacity; j++) {
                int k;
                for (k=i+1; k<n; k++) {
                    if (frames[j] == pages[k]) {
                        if (k > farthest) {
                            farthest = k;
                            pos = j;
                        }
                    }
                }
                if (k == n) {
                    pos = j;
                    break;
                }
            }
            if (pos == -1) {
                pos = 0;
            }
            frames[pos] = pages[i];
            faults++;
        }
    }
    printf("Pages %d -> ", pages[i]);
    for (int j=0; j<capacity; j++) {
        if (frames[j] != -1) {
            printf("%d ", frames[j]);
        }
        else {
            printf("- ");
        }
    }
    printf("\n");
    printf("Total Page Faults = %d\n", faults);
}

```



```

int main() {
    int pages(N), n, capacity;
    printf("Enter number of pages: ");
    scanf("%d", &n);
    printf("Enter page reference string: ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &pages[i]);
    }
    printf("Enter frame capacity: ");
    scanf("%d", &capacity);
    FIFO(pages, n, capacity);
    LRU(pages, n, capacity);
    Optimal(pages, n, capacity);
    return 0;
}
    
```

Output :-

Enter number of pages: 12

Enter page reference string:

1 2 3 4 1 2 5 1 2 3 4 5

Enter frame capacity: 3

FIFO Page Replacement

Page 1 → 1 - -

Page 2 → 1 2 -

Page 3 → 1 2 3

Page 4 → 4 2 3

Page 1 → 4 1 3

Page 2 → 4 1 2

Page 5 → 5 1 2

Page 1 → 5 1 2

Page 2 → 5 1 2

Page 3 → 5 3 2

Page 4 → 5 3 4

Page 5 → 5 3 4

Total Page Faults = 9

LRU Page Replacement

Page 1 → 1 - -

Page 2 → 1 2 -

Page 3 → 1 2 3

Page 4 → 4 2 3

Page 1 → 4 1 3

Page 2 → 4 1 2

Page 5 → 5 1 2

Page 1 → 5 1 2

Page 2 → 5 1 2

Page 3 → 3 1 2

Page 4 → 3 4 2

Page 5 → 3 4 5

Total Page Faults = 10.

Optimal Page Replacement

Page 1 → 1 - -

Page 2 → 1 2 -

Page 3 → 1 2 3

Page 4 → 1 2 4

Page 1 → 1 2 4

Page 2 → 1 2 4

Page 5 → 1 2 5

Page 4 → 1 2 5

Page 2 → 1 2 5

Page 3 → 3 2 5

Page 4 → 4 2 5

Page 5 → 4 2 5

Total Page Faults = 7.

26/5/26